

Feature comparison: LangSmith vs Langfuse

A decision document. What each platform actually does, where they overlap, and where the difference is real rather than cosmetic.

Deployment and hosting features are covered briefly at the end. They matter less for this decision than the observability and analysis capabilities.

The one-paragraph version

Both tools solve tracing and evaluation to a comparable standard. If that is all you need, the decision comes down to hosting, not features.

They diverge in two places, and both are worth understanding before choosing:

1. **LangSmith has automated analysis that Langfuse does not.** Insights and Engine read your production traces and tell you what is wrong, rather than waiting for you to go looking. Engine goes further and proposes the code fix.
2. **Langfuse can be self-hosted, for free, with no feature loss.** Every core capability in this document is in the MIT-licensed version.

That is the trade. Automated analysis on one side, data ownership on the other.

Feature matrix

Grouped by what you would actually use it for.

Observability

Capability	LangSmith	Langfuse	Notes
Distributed tracing	Yes	Yes	Both capture the full agent tree
Automatic instrumentation	Yes	No	LangSmith needs one env var; Langfuse needs a handler per call site
Token and cost per span	Yes	Yes	Both roll costs up to the trace
Waterfall timing view	Yes	No	Langfuse shows durations but no timeline chart
Error propagation up the tree	Partial	Yes	Langfuse marks every ancestor span; LangSmith marks the root and gives a traceback
Rendered agent graph	No	Yes	Langfuse draws the LangGraph structure
Grouping runs into conversations	Threads	Sessions	Equivalent
Per-user cost attribution	Metadata only	Dedicated page	Langfuse aggregates spend per end user
Environment separation	Yes	Yes	
Filter and search over traces	Yes	Yes	Both support structured filters

Analysis and monitoring

Capability	LangSmith	Langfuse	Notes
Dashboards and charts	Yes	Yes	Both ship prebuilt boards
Custom dashboards	Yes	Yes	
Error rate over time	Yes	Yes	
Threshold alerting	Alerts	Monitors	Equivalent
Automatic failure clustering	Insights	No	No Langfuse equivalent
Automated root cause and fix	Engine	No	No Langfuse equivalent
Rule-based automations on traces	Yes	Partial	See below

Evaluation

Capability	LangSmith	Langfuse	Notes
Golden datasets	Yes	Yes	
Offline experiments	Yes	Yes	
LLM-as-judge in code	Yes	Yes	
LLM-as-judge configured in the UI	Yes	Yes	Langfuse's is aimed at continuous scoring of live traffic
Judge returns an explanation	Roll your own	Built in	Langfuse's <code>Evaluation</code> object has a <code>comment</code> field
Error rate shown next to score	Yes	Partial	LangSmith puts an error percentage in the experiment table
Experiment comparison charts	Yes	Yes	LangSmith's are richer
Human annotation queues	Yes	Yes	Equivalent
Pairwise experiments	Yes	No	

Prompts and iteration

Capability	LangSmith	Langfuse	Notes
Prompt management and versioning	Yes	Yes	
Prompt playground	Yes	Yes	
Protected or labelled prompt versions	Yes	Enterprise licence	Free in Langfuse Cloud, gated in self-hosted
Visual agent debugger	Studio	No	The single biggest gap. See below
Context Hub	Yes	No	

Governance and data

Capability	LangSmith	Langfuse	Notes
Self-hosting	Enterprise only	Free, MIT	The decisive difference for regulated work
Data never leaves your network	Enterprise only	Yes	
RBAC	Enterprise	Enterprise licence	Paid on both sides
Audit logs	Enterprise	Enterprise licence	Paid on both sides
SSO	Enterprise	Yes	
Data masking	Enterprise	Enterprise licence	
Retention policies	Plan-based	Enterprise licence	Self-hosted OSS keeps data until you delete it

Build and run

Included for completeness. These are LangSmith moving into territory Langfuse has not entered.

Capability	LangSmith	Langfuse
Agent hosting	Deployments (paid)	No
Code sandboxes	Sandboxes (paid)	No
LLM proxy and gateway	LLM Gateway (beta)	No

Where LangSmith is genuinely ahead

Insights: automatic failure clustering

Insights reads your traces and categorises them on its own. Instead of you filtering for errors and reading twenty of them, it produces a hierarchy of categories and subcategories with error rate, latency, cost and evaluator scores aggregated per category, plus an executive summary with prevalence percentages and links to representative traces.

You configure it by answering questions in plain language, or by specifying filters and categories manually. Jobs run on a schedule: daily, weekly or custom.

Why it matters: at 300 traces a day nobody reads them all. Insights is how you find out that 12% of your sessions fail on the same tool call without anyone noticing.

What it costs: Plus and Enterprise plans only. Roughly **\$1 to \$2 per 1,000 threads** with OpenAI models, or **\$3 to \$4** with Anthropic models.

Engine: automated diagnosis and repair

Engine goes further than Insights. It analyses production traces, groups related failures, triages by severity, and for each issue summarises the failure mode and identifies what needs to change. It then **writes the prompt or code fix**. Connect your repository and it can open a GitHub pull request ready for review.

It also writes tests so the issue does not come back, suggests online evaluators to confirm the fix held, and recommends examples to add to your offline datasets.

Why it matters: this is the difference between a tool that shows you problems and one that closes the loop. There is nothing comparable in Langfuse or, honestly, in most of the market.

What it costs: metered in LangChain Compute Units at **\$1.50 per LCU**. Consumption depends on how many traces it analyses and how deep the analysis goes. See the caveat in [07-cost-model.md](#): LangChain publishes no LCU-per-trace figure, so this line is genuinely hard to forecast.

Automations

Rules that fire on incoming traces: sample a percentage of runs matching a filter and route them somewhere useful, such as an annotation queue for human review, a dataset, or an online evaluator. This is how you build a continuous quality loop rather than running evals only when you remember to.

Langfuse covers part of this through UI-configured evaluators with sampling rates, but does not offer the same general trigger-and-action model.

Studio

A visual debugger for LangGraph agents. It draws your graph, runs it step by step, and lets you inspect and **edit state mid-run** before continuing. It connects to a LangSmith Deployment or to a graph running locally.

Langfuse can render a graph of a run that already finished. Studio lets you drive one while it executes. Different jobs entirely. If your team builds on LangGraph, this alone is a reason to keep a LangSmith seat even if traces live elsewhere.

Where Langfuse is genuinely ahead

Self-hosting, free and complete

This is the headline. Langfuse's own documentation states that **all core features and APIs are available in the MIT-licensed version without any limits**, on the same deployment infrastructure Langfuse Cloud runs on.

Only governance extras need a paid licence key: project-level RBAC, protected prompt labels, data retention policies, audit logs, server-side data masking, UI customisation, SCIM, and the org and instance management APIs.

On LangSmith, self-hosting is **Enterprise tier only**. For a team that cannot send trace data to a third party, that is not a feature comparison, it is a shortlist of one.

Per-user cost attribution

Langfuse has a Users page that aggregates events, tokens and spend per end user. In this lab it showed user `vikash` at 102 events, 6.36K tokens and \$0.02.

LangSmith stores `user_id` as metadata you can filter on, which answers "show me this user's traces" but not "what does this customer cost us". If you are billing customers or investigating a heavy account, that difference is real.

Error visibility inside the trace

Covered in detail in [01-tracing.md](#). Langfuse marks every span the error passed through, so you see the failure's path through the agent at a glance. LangSmith marks the root and gives you a Python traceback to read.

Small thing, used constantly.

Judge explanations by default

Langfuse evaluators return an `Evaluation` object with a `comment` field shown next to the score, so a failing grade tells you why. LangSmith evaluators return a bare boolean; you can return a dict to work around it, but the default is a number with no reasoning attached.

Where they are genuinely equivalent

Do not let a vendor deck tell you otherwise. Both do these to a comparable standard:

- Distributed tracing with token and cost accounting
- Golden datasets and offline experiments
- LLM-as-judge evaluation
- Human annotation workflows
- Prompt management and versioning
- A prompt playground
- Dashboards and threshold alerting
- Grouping runs into conversations

If your requirements stop here, pick on hosting and price, not on capability.

Recommendation

If this is true of your team	Choose
Trace data cannot leave your network	Langfuse self-hosted. Nothing else qualifies below Enterprise pricing
You build on LangGraph and debug agents daily	LangSmith. Studio has no equivalent
You want failures found and fixed automatically	LangSmith. Insights and Engine are the differentiators
You need per-customer cost attribution	Langfuse
You have many engineers who need access	Langfuse. Cloud has unlimited users; LangSmith is \$39 per seat
You just need tracing and evals, small team	Either. Decide on price, see 07-cost-model.md

Running both is a legitimate option and cheaper than it sounds. With both sets of credentials present, a single run lands in both tools. A team could self-host Langfuse as the system of record for all traces, and keep a small number of LangSmith seats for Studio and Engine during development.

A note on evidence

Everything about tracing, evaluation, dashboards, datasets, sessions, users and error handling in this document was verified by running both tools on identical code. The screenshots are in [01-tracing.md](#), [02-evaluation.md](#) and [03-features.md](#).

Insights, Engine and Automations were not tested. They are Plus and Enterprise features and this lab runs on the Developer plan. Those three sections are drawn from LangChain's official documentation and are marked as such rather than dressed up as first-hand findings.

Sources

- [LangSmith pricing](#)
 - [LangSmith Insights documentation](#)
 - [LangSmith Engine](#)
 - [Langfuse pricing](#)
 - [Langfuse self-hosting licence](#)
-